The Journal of Open Source Software

YASMOT: Yet another stereo image multi-object tracker

Ketil Malde [1,2]

1 Institute of Marine Research, Bergen, Norway **2** Department of Informatics, University of Bergen, Norway

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 25 April 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

There now exists many popular object detectors based on deep learning that can analyze images and extract locations and class labels for occurrences of objects. For image time series (/i.e./, video or sequences of stills), tracking objects over time and preserving object identity can help to improve object detection performance, and is necessary for many downstream tasks, including classifying and predicting behaviors, and estimating total abundances. Here we present `yasmot`, a lightweight and flexible object tracker that can process the output from popular object detectors and track objects over time from either monoscopic or stereoscopic camera configurations. In addition, it includes functionality to generate consensus detections from ensembles of object detectors.

Statement of need

`yasmot` is a multi-object tracker, implemented in Python, and available under a GPLv2 license. In addition to tracking objects over time, it can link observations between left and right cameras in a stereo configuration, which further improves detection performance, and allows extracting depth information and estimate the sizes of objects. It has been tested on the output from RetinaNet (Lin et al., 2018) the YOLO family (Redmon et al., 2016) of object detectors.

In contrast to more complex approaches that rely on analyzing the image contents (cf. Related Work, below), `yasmot` works with detections only, reading observations from a separate object detector, and linking them across time based on the relative position and dimensions of bounding boxes and on the classification labels and confidence scores. As a result, `yasmot` is a fast, lightweight alternative with few dependencies.

Tracks are calculated by calculating distances between detections in two frames, and finding an optimal pairing using the Hungarian algorithm (Kuhn, 1955). Distances are calculated by applying a Gaussian to detection parameters (i.e., position and size coordinates) separately. In contrast to IoU-based distances, the use of Gaussian distances allows detections to be connected even if non-overlapping, which is important for low frame rates and between stereo frames of objects close to the camera. The sharpness of the Gaussian is controlled by the `--scale` parameters (see below). The Hungarian algorithm solves assignment on a bipartite graph, so it can only work on two frames at a time. It is possible to generalize it to consider multiple simultaneous frames (for instance, when calculating consensus from multiple detectors or tracking stereo images over time), but the computational cost become prohibitive and a heuristic is used instead.

Usage and options

Controlling tracking sensitivity

The `--scale` parameter controls how the different bounding box pairs are ranked when considered for tracking (or stereo matching). The algorithm uses a Gaussian score for position and size, and this parameter controls the sharpness (or temperature) of the Gaussian. Generally, if you have large changes between frames (rapidly moving objects or low frame rate, you can try reducing this parameter.

Handling missing detections

Object detectors are sometimes unreliable, and objects may fail to be detected, especially when they are partially occluded or otherwise difficult to identify. To remedy this, `yasmot` will attempt to link tracks across missing detections. The maximum gap allowed in a track is controlled by the parameter `--max_age`. The age is derived from the frame name, and unless the frame name is a plain number, the extraction can be specified with `--time_pattern`.

If desired, the missing observations can be inferred by specifying the `--interpolate` option. The interpolated detections can be identified from their probability, which is set to 0.0000.

In case there are classes representing an unknown or unidentified object, it is possible to specify the label with the `--unknown` parameter to avoid having this class be called as a consensus class for a track.

Tracking stereo images

The `-s` option links objects taken with a stereoscopic camera setup. Normally tracks will be generated, but the `--no-track` option can be specified to only link detections between the cameras, and not in time.

Ensemble predictions

If you run multiple object detectors, it may be useful to combine the outputs into a consensus set of predictions. This can be achieved by specifying the `-c` option. Again you can use `--no-track` if you just want the frame-by-frame consensus and not perform tracking as well.

Using pixel-based coordinates

The YOLO object detector (Redmon et al., 2016) outputs image coordinates as fractional images, i.e. values in the range from 0 to 1. Other object detectors, like RetinaNet (Lin et al., 2018), may output a CSV file with pixel-based coordinates. Since `yasmot` does not require the images to be available, you therefore may need to specify the pixel size of the images, e.g. as `--shape 1228,1027` when using pixel-based coordinates.

Output formats

If the `-o` option is not specified, the output will be written to `stdout`. If the option is specified as `-o outfile`, the following files may result, depending on the selection of other parameters specified:

- `outfile.frames` - frame annotations in YOLO format, with track number added. For stereo images, each line will contain the bounding boxes for both images.
- `outfile.tracks` - the list of tracks with the sequence of observations that constitute each track.
- `outfile.pred` - per track consensus class predictions.

DRAFT

Examples

The following examples are taken from the included test suite and the data files can be found in the `tests` directory.

Perform tracking on a directory of predictions from YOLO

This reads a directory with a text file of annotations for each frame:

```
yasmot tests/lab2
```

Perform tracking with interpolation

Interpolation creates virtual annotations to fill in gaps (i.e., missing detections) in the tracks:

```
yasmot tests/lab2 --interpolate
```

Perform tracking limiting gap size

Here, we only connect tracks with a maximum of two frames without detections. The frames in `tests/lab` are named `frame_000152.txt`, `frame_000153.txt`, and so on, and the `--time_pattern` expression must match this format.

```
yasmot --max_age 2 --time_pattern frame_\{:d\}.txt tests/lab2
```

Perform tracking on stereo images

The `-s` option requires the user to specify two input stream, with the assumption that the left images are specified before the right images. Here we process predictions in pixel-based CSV format, and thus we must specify the images size with `--shape`:

```
yasmot -s --shape 1228,1027 tests/stereo1_Left.csv tests/stereo1_Right.csv
```

Only link stereo predictions without tracking

Again we use pixel-based RetinaNet predictions between the two cameras:

```
yasmot -s --no-track --shape 1228,1027 tests/stereo1_Left.csv tests/stereo1_Right.csv
```

Merge predictions from multiple object detectors

Here we use predictions from a family of YOLO v8 models to provide ensemble predictions:

```
yasmot -c tests/consensus/y8x*
```

Related work

Tracking objects has long been recognized as a fundamental task in computer vision, with applications ranging from surveillance to autonomous systems. The widely-used image processing library OpenCV has incorporated several algorithms and components dedicated to object tracking, reflecting the task's importance. These include BOOSTING, MIL, KCF, CSRT, MedianFlow, TLD, MOSSE, and GOTURN (Bradski, 2000). Each of these methods offers distinct approaches to balancing speed, accuracy, and robustness, catering to a variety of real-time tracking needs. Another commonly used tool is SORT (Simple Online and Realtime Tracking) (Bewley et al., 2016), which uses uses a Kalman filter to predict object motion and associates predictions with detections using Intersection over Union (IoU).

The advent of deep learning object detectors like YOLO have brought new object tracking tools to the fore, and the popular implementations of YOLO by Ultralytics (Jocher et al., 2023) integrate two such tracking algorithms, ByteTrack and BoT-SORT. Like yasmot, ByteTrack processes object detection model output to associate bounding boxes across frames, but it uses intersection over union (IoU) instead of Gaussian distances to link detections. BoT-SORT

DRAFT

(Aharon et al., 2022), on the other hand, extends SORT by incorporating additional motion and appearance cues to enhance tracking precision.

Other object trackers that examine the detected objects to support tracking Tracktor++ (Bergmann et al., 2019), and DeepSORT (Wojke et al., 2017), which similarly to BoT-SORT extends SORT with features from a deep learning model to matches detections across frames more reliably, particularly in crowded or dynamic environments.

Acknowledgments

This work was developed using data from the CoastVision (RCN 325862) and CRIMAC projects (RCN 309512), and after productive discussions with Vaneeda Allken, Taraneh Westergerling, and Peter Liessem.

References

Aharon, N., Orfaig, R., & Bobrovsky, B.-Z. (2022). BoT-SORT: Robust associations multi-pedestrian tracking. *arXiv Preprint arXiv:2206.14651*.

Bergmann, P., Meinhardt, T., & Leal-Taixe, L. (2019). Tracking without bells and whistles. *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 941–951.

Bewley, A., Ge, Z., Ott, L., Ramos, F., & Upcroft, B. (2016). Simple online and realtime tracking. *2016 IEEE International Conference on Image Processing (ICIP)*, 3464–3468.

Bradski, G. (2000). The OpenCV Library. *Dr. Dobb's Journal of Software Tools*.

Jocher, G., Chaurasia, A., & Qiu, J. (2023). *Ultralytics YOLOv8* (Version 8.0.0). https://github.com/ultralytics/ultralytics

Kuhn, H. W. (1955). The hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, *2*(1-2), 83–97.

Lin, T.-Y., Goyal, P., Girshick, R., He, K., & Dollár, P. (2018). *Focal loss for dense object detection*. https://arxiv.org/abs/1708.02002

Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You only look once: Unified, real-time object detection. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.

Wojke, N., Bewley, A., & Paulus, D. (2017). Simple online and realtime tracking with a deep association metric. *2017 IEEE International Conference on Image Processing (ICIP)*, 3645–3649.

DRAFT